AutonomousDR — 36-Week Build Plan

Autonomous Multi-Cloud Disaster Recovery System

Faculty of IT & CS, University of Central Punjab · Fall 2026 – Spring 2027

Supervisor: Misbah Naz

The Strategy in One Paragraph

Design once, up front, completely. Then build incrementally through that fixed design. Weeks 1–2 produce no features at all — they produce five frozen interfaces and the complete database schema, including tables that stay empty until month four. Everything after that is thickening, never replacing. This is what separates "incremental building" (good) from "incremental design" (the thing that causes month-six rewrites).

Capacity Model

Basis	Figure
Nominal	5 h/day × 6 days × 3 people = **90 person-hours/week**
Over 36 weeks	3,240 person-hours nominal
Realistic after coursework, exams, Ramadan, learning curve, illness	**~1,700 productive hours**
Documentation (SRS, SDS, thesis) — non-negotiable	~250 hours of that
Net build time	**~1,450 hours**

That is enough for essentially all of slide 9. It is **not** enough for slide 9 plus two research dead ends. Hence the decision gates below.

Role Ownership

Three roles, one person each. Every task tag in this plan refers to a **role**, not a person — if someone swaps role mid-project, the plan does not change.

Role	Owns
BACKEND	Backend services, database, infrastructure, cloud provider integrations, Terraform, consensus layer, Celery, CI/CD, deployment. **Also owns integration** — merges to `main` every Friday whether modules are finished or not.
FRONTEND	Dashboard, WebSocket layer, charts, topology views, UX, report exports, documentation formatting and diagrams.
AI	Anomaly detection models, GNN, RAG pipeline, decision logic and utility function, evaluation harness, dataset generation, results chapter.
ALL	Weeks 1–2 in full, every Friday integration session, all gate checks, demos and defence prep.

Nobody works alone on their island for six weeks. Rotate one pair-programming session per week so every role has a backup.

Non-Negotiable Ground Rules

1. **The design is frozen after Week 2.** Changing an interface after that requires an ADR and all three agreeing in writing. Adding an *implementation* of an existing interface is free and expected.

2. `main` **always demos.** If it doesn't run end to end, fixing that is the only priority.
3. **Friday is integration day.** Every branch merges Friday. No branch lives longer than one week.
4. **The collector runs 24/7 from Week 3.** Your dataset accumulates while you sleep. This is why training in Week 10 is easy.
5. **Every metric goes in the results table, including bad ones.** Failed experiments are findings. Hidden failures are how you get caught in the viva.
6. **One rest day per week, actually taken.** Week-30 burnout is the most common way ambitious FYP teams collapse.

Decision Gates

Check these on the stated Friday. Each has a pre-agreed consequence so you never have to argue about it under pressure.

Gate	Week	Test	If FAILED
A	6	Full loop works on simulator: fault → detect → decide → cost-check → act → log → dashboard	Cut GCP, Thompson Sampling and PBFT from scope **now**. Do not defer the decision.
B	12	Real AWS read path live AND Isolation Forest beats the threshold baseline on F1	Drop the GNN. Ship NetworkX reachability scoring instead (Week 13 output only).
C	17	Detection + consensus + RAG + explanations all working together	Cut Phase 6 items in this order: Thompson → PBFT → OSINT → GCP
D	30	—	**Absolute feature freeze.** No new features after this date under any circumstances. Only bug fixes, evaluation, and writing.

Phase Overview

Phase	Weeks	Dates	Theme
0	1–2	Aug 17 – Aug 30	Architecture freeze — zero features
1	3–6	Aug 31 – Sep 27	The spine — one path through every layer
2	7–12	Sep 28 – Nov 8	Real clouds + real detection
3	13–17	Nov 9 – Dec 13	Depth — graph, RAG, consensus
4	18–19	Dec 14 – Dec 27	Milestone 1 + semester close
5	20–21	Dec 28 – Jan 10	Break sprint — no classes, maximum hours
6	22–25	Jan 11 – Feb 7	Hard mode — the ambitious extras
7	26–30	Feb 8 – Mar 14	Hardening + evaluation (Ramadan — desk work)
8	31–32	Mar 15 – Mar 28	Final evaluation + polish
9	33–36	Mar 29 – Apr 25	Thesis + defence

> Adjust all dates to UCP's actual academic calendar. Ramadan is estimated at approximately Feb 8 – Mar 9, 2027 with Eid al-Fitr around Mar 10–12 — confirm and shift Phase 7 accordingly.

PHASE 0 — ARCHITECTURE FREEZE

Week 1 · Aug 17 — Foundations

> **Goal:** Every team member can run the full infrastructure stack locally, and the architecture is decided on paper.

- ALL — Monorepo: `core/`, `api/`, `web/`, `ml/`, `infra/`, `docs/`, `corpus/`. Branch protection on `main`, PR required.
- BACKEND — `docker-compose.yml` bringing up Postgres 16 (+ pgvector), Redis, InfluxDB, Celery worker. Databases in Docker; app code runs natively with hot reload.
- ALL — Write 8 ADRs in `docs/adr/`: provider abstraction, Postgres + InfluxDB split, pgvector over a separate vector DB, quorum-before-PBFT, RAG-not-fine-tuning, Celery for async, simulator-first testing, time-based evaluation splits.
- FRONTEND — Vite + React + Tailwind scaffold, WebSocket client stub, component library decision.
- AI — Read and annotate the three core papers (Li et al. utility function, Liu et al. consensus, Karami et al. anomaly detection). Extract exact formulas into `docs/formulas.md`.
- ALL — Set up Jira with all 36 weeks as epics. Yes, all of them, now.

> **Done when:** `docker compose up` works on all three laptops and the 8 ADRs are committed.

Week 2 · Aug 24 — The Freeze

> **Goal:** Five interfaces and the complete schema exist. No implementations. This is the most important week of the project.

- BACKEND — Define all five abstract interfaces in `core/interfaces/` with complete type-hinted signatures and docstrings, no bodies:
  - `CloudProvider` — `get_health()`, `list_resources()`, `estimate_cost(action)`, `execute(action)`, `get_regions()`. Designed so AWS, Azure, GCP and Terraform all fit.
  - `AnomalyDetector` — `fit(X)`, `score(window) -> float`, `explain() -> dict`. Threshold, IForest, LSTM and GNN must all satisfy it.
  - `ConsensusStrategy` — `propose(decision)`, `vote(proposal)`, `commit()`. Quorum today, PBFT in Week 24, same signature.
  - `ActionExecutor` — `dry_run(action)`, `execute(action)`, `rollback(action_id)`, `tier` property.
  - `ExplanationEngine` — `explain(context) -> str`. Hosted LLM, local LLM, and dumb template all satisfy it.
- BACKEND — **Complete** SQL schema + Alembic migration: `tenants`, `resources`, `incidents`, `detections`, `proposals`, `votes`, `actions`, `cost_estimates`, `audit_log` (hash-chained), `graph_nodes`, `graph_edges`, `corpus_chunks` (pgvector), `bandit_state`, `risk_signals`, `savings_reports`. Most sit empty for months. That is the point.
- AI — Sequence diagrams for the three critical paths: detection→action, consensus vote, RAG explanation.
- FRONTEND — OpenAPI contract for every endpoint the frontend will ever need, including Phase 6 ones. Mock server returns fixtures.
- ALL — **Design review with the supervisor.** Walk her through the interfaces and the scope split (committed vs stretch). Get the PBFT-as-pluggable and RAG-not-fine-tuning decisions explicitly blessed.

> **Done when:** interfaces committed, migration runs clean, advisor has signed off. **After this Friday the design is frozen.**

PHASE 1 — THE SPINE

Week 3 · Aug 31 — Simulated Cloud + Collector

> **Goal:** Fake telemetry flowing into InfluxDB continuously.

- BACKEND — `SimulatedProvider` implementing `CloudProvider`: 8 resources across 3 fake regions, 2 fake tenants. Realistic metrics — daily seasonality, noise, configurable fault injection via API.
- BACKEND — Collector: Celery beat task polling every 10s, writing to InfluxDB with tenant and region tags.
- AI — Metric schema decision: which 20 signals you collect. Changing this later invalidates your dataset, so decide it once, properly.
- FRONTEND — Dashboard shell: layout, routing, theme, empty-state components.

> **Done when:** the collector has been running for 24 hours uninterrupted and InfluxDB has ~8,600 rows per stream. **Do not stop it again for the rest of the project.**

Week 4 · Sep 7 — Detection Stub + Audit Trail

> **Goal:** A dumb rule creates real incidents, and everything is logged tamper-evidently.

- AI — `ThresholdDetector` implementing `AnomalyDetector`: 3 consecutive breaches → detection. Deliberately simple. **This is your permanent baseline — never delete it.**
- BACKEND — Incident lifecycle: detection → `incidents` row → state machine (open/deciding/acting/resolved/failed).
- BACKEND — Hash-chained `audit_log`: each row stores the SHA-256 of the previous row. Write a `verify_chain()` function now.
- FRONTEND — Live incident list on the dashboard, polling for now.

> **Done when:** you trigger a simulator fault, an incident appears in Postgres, and `verify_chain()` returns true.

Week 5 · Sep 14 — Decision Engine + Cost Gate

> **Goal:** The system chooses an action and refuses to overspend.

- AI — Implement `U(c,l,m)` exactly as in `docs/formulas.md`. Hardcode weights. **Unit-test it against the paper's own worked values** — this is your correctness proof.
- BACKEND — Cost estimator with a static price table behind a `PricingSource` interface (live APIs swap in at Week 12).
- BACKEND — Budget gate: monthly cap per tenant, pre-action estimate written to `cost_estimates` *before* execution, hard refusal above cap with a logged reason.
- BACKEND — `QuorumConsensus` implementing `ConsensusStrategy`, single-node for now, real voting in Week 17.
- BACKEND — `SimulatedExecutor` — Tier 1 actions only (DNS weight shift, traffic reroute).

> **Done when:** an incident produces a scored decision, a cost estimate, a vote record, and an executed action — all six tables populated in one flow.

Week 6 · Sep 21 — Dashboard + GATE A

> **Goal:** The whole loop, visible, on screen. This is the week that decides your project.

- FRONTEND — WebSocket push from FastAPI. Live metric charts (Recharts), incident feed, action timeline, cost counter. Ugly is fine. Working is not optional.
- FRONTEND — Fault injection button on the dashboard for demos.
- ALL — End-to-end integration. Record a 90-second screen capture of the full loop.
- ALL — **GATE A CHECK (Friday).** Fault → detect → decide → cost-check → vote → act → log → visible, under 60 seconds.

> **Done when:** the video exists. Send it to your advisor. You now have a demoable project in Week 6, and everything from here is improvement rather than hope.

PHASE 2 — REAL CLOUDS + REAL DETECTION

Week 7 · Sep 28 — AWS, Read-Only

Goal: Real cloud data behind the same interface.

- BACKEND — `AWSProvider` implementing all five `CloudProvider` methods. Free tier: one t-class EC2, CloudWatch metrics, Route 53 health check. **Read paths only — no write actions yet.**
- BACKEND — Billing alarm at $5 on the AWS account. Do this before anything else.
- BACKEND — Secrets handling: `.env` locally, GitHub Secrets in CI, never a key in the repo. Add pre-commit hook to block key patterns.
- FRONTEND — Provider badges in the UI; simulated and real resources side by side.
- AI — Begin SRS draft (functional requirements from the frozen OpenAPI contract).

Done when: the dashboard shows a real EC2 instance's CPU next to simulated resources, and switching providers is a config change.

Week 8 · Oct 5 — The Data Factory

Goal: An automated labelled-failure generator. This permanently solves your dataset problem.

- BACKEND — Docker microservice testbed (Online Boutique or Sock Shop) + load generator + OpenTelemetry → Prometheus.
- AI — Fault injection script looping: 10 min normal traffic → random fault (`docker stop` , `docker pause` , `tc netem` 500ms latency, CPU hog) → record exact start/end/type/target to `labels.jsonl` → recover → repeat.
- AI — Episode export: aligned metric windows + labels, versioned in `ml/datasets/v1/` .
- FRONTEND — SRS document assembly and formatting.

Done when: an overnight run produces 30+ labelled episodes with exact ground truth. Schedule it to run every night from now on.

Week 9 · Oct 12 — Azure + Normalisation

Goal: Two real clouds, one data model.

- BACKEND — `AzureProvider` — Azure Monitor metrics, free tier resources.
- BACKEND — Metric normalisation layer: AWS, Azure and simulated metrics map onto your 20 canonical signals. This is genuinely fiddly; the interface is what makes it contained.
- BACKEND — Cross-provider health aggregation and region mapping.
- FRONTEND — Multi-cloud topology view.
- AI — Dataset now at ~200k timesteps. First exploratory analysis notebook.

Done when: one dashboard shows AWS + Azure + simulated resources with comparable metrics.

Week 10 · Oct 19 — Isolation Forest + Benchmark Harness

> **Goal:** Real ML in the loop, and a rig that makes every future comparison cheap.

- AI — `IsolationForestDetector` implementing `AnomalyDetector`, trained on 6 weeks of collected normal telemetry.
- AI — **Evaluation harness** — the most valuable thing you build this week. Time-based split (first 70% / last 30%, never shuffled). Reports precision, recall, F1, false-positive rate, detection delay in seconds. Same harness for every detector forever.
- AI — PyOD benchmark: IForest vs LOF vs ECOD vs COPOD vs autoencoder on identical splits. Five extra rows for one afternoon's work.
- BACKEND — Detector hot-swap via config; both threshold and IForest runnable in shadow mode simultaneously.
- FRONTEND — Anomaly score visualisation with the detection threshold overlaid.

> **Done when:** a results table compares 6 detectors on your own data, and the pipeline runs on IForest in production while the threshold rule shadows it.

Week 11 · Oct 26 — LSTM

> **Goal:** Forecast-residual detection, self-labelling.

- AI — `LSTMDetector`: 2 layers, 64 hidden units, 60-step input window, predicts next timestep. Residual magnitude becomes the anomaly score. Trains on CPU in under 30 minutes.
- AI — Ensemble: IForest + LSTM with configurable combination, tuned for false-positive rate since your proposal commits to near-zero false actions.
- AI — Comparison table v1, all 7 detectors, committed to `docs/results/`.
- BACKEND — Model registry: versioned artefacts, metadata, which model produced which detection recorded in `detections`.
- BACKEND — Nightly retrain Celery task with automatic rollback if validation F1 regresses.

> **Done when:** the ensemble beats the threshold baseline on F1 *and* on false-positive rate, with numbers in the repo.

Week 12 · Nov 2 — Live Pricing + First Real Action + GATE B

> **Goal:** Real money, real infrastructure changes.

- BACKEND — Live `PricingSource` implementations: AWS Price List API, Azure Retail Prices API. Cached, with static fallback.
- BACKEND — `AWSExecutor` Tier 1: real Route 53 weighted-record failover. Dry-run mode first, then live, on throwaway resources.
- BACKEND — Cost reconciliation: estimated vs actual, variance tracked per action type.
- AI — SDS / design document draft.
- ALL — **GATE B CHECK (Friday).**

> **Done when:** you break a real AWS health check and the system performs a real DNS failover, with a real cost estimate, logged and verified.

PHASE 3 — DEPTH

Week 13 · Nov 9 — Dependency Graph

Goal: Cascade awareness. No training required.

- AI — Extract the service dependency graph from OpenTelemetry traces → `graph_nodes` / `graph_edges`.
- AI — NetworkX reachability and blast-radius scoring; feed impact score into `U(c,l,m)`. **This is the GNN fallback, built first deliberately.**
- FRONTEND — Interactive topology graph with live health colouring — visually the most impressive thing on your dashboard.
- BACKEND — Graph snapshot versioning so historical incidents keep their contemporaneous topology.

Done when: killing one service shows the correct downstream blast radius on screen, and impact score changes the chosen action.

Week 14 · Nov 16 — GNN

Goal: Learned cascade prediction.

- AI — PyTorch Geometric GCN/GraphSAGE, node classification (will this service fail in the next N minutes), trained on labelled episodes. Free Kaggle GPU.
- AI — Compare against Week 13's NetworkX baseline in the same harness. **If it doesn't win, keep NetworkX and report the negative result** — that is a legitimate finding, not a failure.
- BACKEND — Inference path: graph snapshot → GNN → risk scores, cached, under 2s.
- FRONTEND — Predicted-cascade overlay on the topology view.

Done when: the GNN is either in production with a benchmark win, or documented as a negative result with the baseline retained.

Week 15 · Nov 23 — RAG

Goal: Retrieval over post-mortems and the tenant's own history.

- AI — Corpus collection into `corpus/`: AWS Post-Event Summaries, Azure PIRs, GCP incident reports, Cloudflare and GitHub post-mortems, provider reliability docs, your own runbooks. Target 150+ documents.
- AI — Chunking, embedding, pgvector index. Global store plus per-tenant store from `audit_log`.
- BACKEND — **Tenant isolation enforced in SQL** — `WHERE tenant_id = :id` inside the retrieval query, never post-filtered. Write the test that proves tenant A never retrieves tenant B's rows.
- AI — Retrieval quality eval: 20 hand-written symptom queries, measure whether the right post-mortem lands in the top 3.
- FRONTEND — Retrieved-sources panel on the incident detail view.

Done when: a simulated latency incident retrieves relevant real post-mortems plus that tenant's own past incidents, and the isolation test passes.

Week 16 · Nov 30 — Explanation Engine

Goal: Plain-English justification for every action.

- AI — `LLMExplanationEngine`: hosted Llama 3 (Groq free tier), prompt assembling current state + decision + retrieved documents. Constrained to cite only provided sources.
- BACKEND — **Off the critical path** — Celery task, stored on the incident row, pushed when ready. Template fallback if the API fails. The action must never wait for the LLM.
- BACKEND — Prompt and response both written to `audit_log` every time.
- AI — Hallucination check: 20 incidents reviewed manually for claims unsupported by retrieved docs. Report the rate honestly.
- FRONTEND — Explanation card in the UI with expandable source citations.

Done when: every incident carries a readable justification, and killing the LLM API doesn't stop recovery.

Week 17 · Dec 7 — Real Consensus + Full Action Tiers + GATE C

Goal: Three voting nodes and the complete action catalogue.

- BACKEND — Three monitor processes, independent detection, 2-of-3 quorum required before any action. Votes recorded in `votes`. Handle disagreement, timeout, split-brain, and node death.
- BACKEND — Action Tiers 2 and 3: scaling, region failover, full DR promotion. Rollback for every action type. Human-approval gate for Tier 3.
- AI — Chaos test the consensus layer: kill one node mid-vote, kill two, partition them.
- FRONTEND — Consensus visualisation — who voted what, and why.
- ALL — **GATE C CHECK (Friday).**

Done when: a single rogue monitor cannot trigger an action, and a killed node doesn't stop legitimate recovery.

PHASE 4 — MILESTONE 1 + SEMESTER CLOSE

Week 18 · Dec 14 — Milestone 1

Goal: Present a working system. Light build load — finals are near.

- ALL — Demo script and rehearsal. Three scenarios: single-service failure, region failure, budget-blocked action.
- ALL — Milestone 1 presentation to the committee.
- AI — SRS and SDS finalised and submitted.
- FRONTEND — Screenshots and diagrams for documentation.
- ALL — Retrospective: what took longer than planned, and adjust Phase 6 scope accordingly.

Done when: Milestone 1 passed and documentation submitted.

Week 19 · Dec 21 — Finals

Goal: Protect your CGPA. **Code freeze.**

- ALL — Exams. No new features. No refactoring.
- BACKEND — Keep the collector and nightly fault-injection running. Data keeps accumulating without you.
- Optional light task: write the README and setup guide.

Done when: exams are done and nothing is broken.

PHASE 5 — BREAK SPRINT

No classes. This is your highest-capacity fortnight of the year — potentially 40+ h/week each. Use it on the two heaviest remaining items.

Week 20 · Dec 28 — Terraform

Goal: Infrastructure-as-code recovery.

- BACKEND — `TerraformExecutor` implementing `ActionExecutor`: HCL modules for DR environments, `plan` as dry-run, `apply` as execution, state in S3 with locking.
- BACKEND — Full-stack recovery action: stand up a complete replica environment in a second region from code.
- BACKEND — Destroy-after-test automation so you never leak billable resources.
- FRONTEND — Terraform plan diff rendered in the UI before approval.
- AI — Thesis Chapter 1 (Introduction) and Chapter 2 (Literature Review) drafted while the context is fresh.

Done when: the system provisions a real DR environment via Terraform and tears it down, cost-tracked.

Week 21 · Jan 4 — GCP + Cross-Cloud Failover

Goal: Three providers.

- BACKEND — `GCPProvider` — Cloud Monitoring metrics, free tier resources.
- BACKEND — True cross-cloud failover: AWS → Azure, AWS → GCP. DNS orchestration via Route 53 and Cloudflare.
- AI — Cross-cloud cost comparison feeding the utility function — this is the core of your "cost-aware" claim, so make it real.
- FRONTEND — Three-cloud unified topology view.

Done when: an AWS region failure triggers failover to a different provider entirely, with the cost trade-off shown in the decision log.

PHASE 6 — HARD MODE

Week 22 · Jan 11 — OSINT Risk Feed

Goal: Predictive geopolitical and provider risk signals.

- BACKEND — Ingest provider status pages/RSS, GDELT or a news API, into `risk_signals`.
- AI — Region risk scoring: recent incident density, geopolitical instability, provider advisory severity. Feeds the utility function as a proactive term.
- AI — Backtest against the March 2026 Middle East event and the October 2025 us-east-1 event: would elevated risk have been visible beforehand? Report honestly either way.
- FRONTEND — Risk heat map by region.

Done when: risk scores measurably shift action selection, with a documented backtest.

Week 23 · Jan 18 — Proof-of-Savings Reporting

Goal: Quantify the value the system delivered. This is your commercial differentiator.

- AI — Savings model: estimated downtime cost avoided, minus action cost, minus system overhead. Document every assumption — a defensible model beats an impressive number.
- BACKEND — `savings_reports` generation, monthly rollups, per-tenant.
- FRONTEND — Executive report view + PDF export.

Done when: the system produces a monthly report with a defensible savings figure and stated assumptions.

Week 24 · Jan 25 — PBFT

Goal: Deliver the ambitious version of consensus — safely, as a plugin.

- BACKEND — `PBFTConsensus` implementing `ConsensusStrategy`: pre-prepare, prepare, commit phases, view change on primary failure, checkpointing. Swappable by config.
- AI — Byzantine test suite: a node that lies, a node that equivocates, a node that stalls.
- AI — Benchmark PBFT vs quorum: latency, message overhead, behaviour under f=1 Byzantine faults. **Report the trade-off honestly — PBFT will be slower, and explaining why that cost buys Byzantine tolerance is exactly the sophisticated answer a committee wants.**
- BACKEND — Quorum remains the default. PBFT is opt-in.

Done when: both strategies pass the same test suite and the comparison is in `docs/results/`.

Week 25 · Feb 1 — Thompson Sampling

Goal: Adaptive utility weights.

- AI — Bandit over utility weight configurations, reward = post-action outcome quality (recovery time, cost variance, whether a follow-up incident occurred). State in `bandit_state`.
- AI — A/B against fixed weights across 100+ simulated incidents.
- BACKEND — Safety rail: learned weights bounded, never able to bypass the budget gate.
- FRONTEND — Weight evolution over time chart.

Done when: you can show whether adaptation beat fixed weights, with a number either way.

PHASE 7 — HARDENING + EVALUATION

Ramadan (approx. Feb 8 – Mar 9). Deliberately front-loaded with desk work — testing, writing, measurement — rather than late-night debugging marathons. Adjust to your actual energy.

Week 26 · Feb 8 — Real Chaos Validation

Goal: Prove it on real infrastructure.

- BACKEND — One controlled AWS Fault Injection Service experiment. Budget it, run it once or twice, capture everything. Billed per action-minute — this is not a development tool.
- AI — Full validation matrix: every fault type × every provider × every action tier. Record recovery time and outcome for each cell.
- AI — Document every failure mode you find. These become your limitations section.

Done when: the validation matrix is complete with measured recovery times, including the real-AWS row.

Week 27 · Feb 15 — Tests + CI/CD

Goal: Engineering credibility.

- ALL — Unit tests to 70%+ on `core/` and `api/`. Integration tests for all three critical paths.
- BACKEND — GitHub Actions: lint, type-check, test, build, on every PR. Branch protection requires green.
- AI — Property-based tests on the utility function and cost gate (Hypothesis).
- FRONTEND — Frontend component tests for the critical views.

Done when: CI is green and required, and coverage is reported in the repo.

Week 28 · Feb 22 — Performance

Goal: Prove the sub-60-second claim under stress, not just when idle.

- BACKEND — Load test: 500 simulated resources, 50 concurrent incidents. Profile and fix the bottlenecks you find.
- AI — Measured latency breakdown per stage: detection, consensus, decision, execution. Publish the distribution, not just the mean — p50, p95, p99.
- BACKEND — InfluxDB retention and downsampling policy; Postgres index tuning.

Done when: p95 end-to-end recovery is under 60 seconds at 10× your demo load, with the breakdown documented.

Week 29 · Mar 1 — Security

Goal: Survive the trust questions.

- BACKEND — Multi-tenant isolation test suite. Every query path verified. This is the question you *will* be asked.

- BACKEND — RBAC, secrets management, credential rotation, least-privilege IAM policies documented.
- AI — Audit-log integrity demo: tamper with a row, show `verify_chain()` catches it. Two minutes of demo that makes you look serious.
- FRONTEND — Approval workflow UI for Tier 3 actions.

> **Done when:** the isolation suite passes and you can demonstrate tamper detection live.

Week 30 · Mar 8 — Eid Week + GATE D

> **Goal:** Light week. Documentation only. **Feature freeze takes effect.**

- FRONTEND — User manual, deployment guide, API documentation.
- AI — Thesis Chapter 3 (Methodology) drafted.
- ALL — **GATE D: absolute feature freeze.** Everything from here is measurement, bug fixing, and writing.

> **Done when:** the freeze is declared in writing and Jira is closed to new features.

PHASE 8 — FINAL EVALUATION + POLISH

Week 31 · Mar 15 — The Numbers

Goal: Generate every figure your thesis will contain.

- AI — Final evaluation run: all 8+ detectors, 300+ labelled episodes, identical time-based splits, full metric set. One reproducible script that regenerates every table and figure.
- AI — Ablation study: what does each component actually contribute? Remove the LSTM, remove the graph, remove RAG — measure the delta. **This is the single most impressive thing in an FYP results chapter.**
- AI — Statistical rigour: multiple seeds, confidence intervals.
- FRONTEND — Final charts and figures at publication quality.

Done when: `make results` regenerates every number in your thesis from raw data.

Week 32 · Mar 22 — Polish

Goal: Make it look like a product.

- FRONTEND — Full UX pass: loading states, error states, empty states, responsive layout, accessibility.
- ALL — Bug bash: three hours, everyone tries to break it, everything found gets triaged.
- FRONTEND — 5-minute demo video, edited, narrated.
- BACKEND — One-command deployment; verify a clean-machine install from the README.

Done when: someone outside the team installs and runs it from your README without asking you anything.

PHASE 9 — THESIS + DEFENCE

Week 33 · Mar 29 — Chapters 4–5

- AI — Chapter 4 (Implementation), Chapter 5 (Results & Evaluation) with all figures from Week 31.
- FRONTEND — Architecture diagrams finalised, screenshots, formatting.
- BACKEND — Appendices: schema, API reference, ADR summary.

Week 34 · Apr 5 — Full Draft

- AI — Chapter 6 (Conclusion & Future Work). Honest limitations section: simulated-infrastructure evaluation, free-tier scale constraints, negative results.
- ALL — Complete draft to the supervisor with time for a real review cycle.
- ALL — Plagiarism check, reference formatting, consistency pass.

Week 35 · Apr 12 — Defence Prep

- ALL — Slide deck. Rehearse the demo **five times**, including once with WiFi deliberately disabled.
- ALL — Q&A prep. Write out answers to the hard ones: Why not a commercial product? Why is Isolation Forest from 2008 acceptable? Why didn't you fine-tune the LLM? How do you know it works without production data? What are the false positives? Why PBFT if quorum is the default?
- FRONTEND — Poster and final report submission.

Week 36 · Apr 19 — Buffer + Defence

- Reserved. Something will overrun. If nothing does, rest — you'll defend better.

Explicitly Out of Scope

State these proactively in your defence. Deliberate exclusions with reasons read as judgement; discovered gaps read as failure.

Excluded	Reason
Fine-tuning Llama 3	No corpus of (incident → correct remediation) training pairs exists, and one cannot be constructed — there are only a few hundred public post-mortems in total. Fine-tuning on that would perform *worse* than RAG at the same task while consuming ~150 hours and GPU budget. RAG is the correct engineering choice, not the humble one.
Production-scale evaluation	No access to production infrastructure. Mitigated by reproducible fault injection with exact ground truth — arguably more rigorous than opportunistic observation of real outages — plus one real AWS FIS validation run.
Kubernetes-native operator	Adds an orchestration layer without adding to the research contribution.
Sub-second recovery	DNS TTL and cloud control-plane latency impose a hard floor. Sub-60-second is the honest, measurable target.

Risk Register

Risk	Likelihood	Mitigation
Cloud bill overrun	Medium	$5 billing alarms from Week 7; destroy-after-test automation; simulator-first for all development
GNN fails to beat baseline	High	NetworkX built first in Week 13; negative result is a publishable finding
PBFT consumes the schedule	Medium	Behind `ConsensusStrategy`, scheduled Week 24 after everything else works, cuttable at Gate C
Team member unavailable	Medium	Weekly cross-pairing; no single-owner knowledge; all decisions in ADRs not heads
Burnout around Week 30	**High**	Mandatory rest day; Phase 7 is desk work during Ramadan; Week 36 is buffer
Advisor requests scope change	Medium	Design review in Week 2 with committed-vs-stretch split blessed in writing
Late integration hell	Low, if rules held	Friday merges from Week 1; `main` always demos

Weekly Cadence

- **Monday, 30 min** — read this week's section aloud together, split the steps, update Jira
- **Wednesday, 15 min** — blocker check only
- **Friday, 2 hours** — merge everything, run the end-to-end demo, tick the "Done when", log actual hours
- **Saturday, 20 min** — retrospective: what slipped, and does next week need adjusting?

Log your actual hours for the first three weeks. If 30 h/week each turns out to be real, all of the above is comfortably within reach. If it's really 18, come back and re-cut Phase 6 early rather than discovering it in March.